1. Introduction and Motivation

1.1 Knowledge Graphs and Their Limitations

Knowledge graphs are graph-based data models where each entities are connected through relationships and they normally utilize query languages to access information (Hogan et al., 2021). Information is represented as a triplet such as:

(Entity$_1$, Relationship, Entity$_2$) e.g. (Barack Obama, President of, United States)

In this way, the static relational information can be represented in a structured format. However, this representation is insufficient for domains where relationships change over time. A static knowledge graph does not capture the fact that some relationships might not remain constant.

1.2 Temporal Knowledge Graphs

Temporal Knowledge Graphs (TKGs) address this issue by introducing time dimension to reflect when a particular event occurred or holds true (Leblay & Chekol, 2018). Therefore, the nodes (entities) and edges (relationships) are annotated by time and the triplet becomes a quadruple:

(Subject, Predicate, Object, Timestamp) e.g. (Nawaz Sharif, Head of Government, Pakistan, 2013-06-05)

In this way the knowledge graph encompasses multiple relationships with similar entities over different time periods without the repetition or cancellation of previous data.

1.3 The ICEWS Dataset and the Near-Duplicate Problem

This property is particularly valuable for event-based datasets such as the Integrated Crisis Early Warning System (ICEWS), a large-scale geopolitical event dataset maintained by Boschee et al. (2015). In this project we specifically used ICEW14, a subset containing over 90,000 geopolitical event quadruples from 2014. The geopolitical information includes information about government, heads of states, military and civilian groups. One prominent thing about the dataset is the existence of near duplicate datapoints across neighboring dates. Here the same entities and relations reside but at different timestamps:

(Protester (Thailand), Threaten to reduce or stop aid, Thailand, 2014-01-04) ← Gold *(Protester (Thailand), Threaten to reduce or stop aid, Thailand, 2014-01-07)*

Here temporal component is the only distinguishing factor between the correct answer and an identical event. This motivates the central research question of this project: how much does temporal information contribute to accurate retrieval compared to semantic information, and can temporal knowledge graph embeddings measurably improve downstream question answering?

1.4 Personal Motivation

As an MSc NLP student, my interest lies in how structured representations such as knowledge graphs can complement purely textual approaches for reasoning. Traditional language models rely on implicit only on semantic depth, whereas knowledge graphs offer a mathematically grounded representation of how entities are related (Hogan et al., 2021). I believe that combining semantic depth with structured graphical vector representation is a powerful direction for NLP. As these systems not only extract meaning from text but also produce a living graphical picture of the dynamic complexity of the dataset. This project placed me directly at that intersection, giving me the opportunity to explore this symbiosis.

1.5 My Role: Encoder Exploration and Integration

My specific contribution was exploring and integrating sentence encoders for semantic re-ranking within the pipeline. Sentence encoders are pretrained transformer based models that map text to dense vectors (Reimers & Gurevych, 2019). Since encoders are trained on natural language, structured quadruples must first be verbalized to be meaningfully embedded. We adopted the following verbalization template:

"{subject} {predicate} {object} on {timestamp}" e.g. "Other Authorities / Officials (Egypt) Criticize or denounce Al Jazeera on 2014-06-24"

Semantic similarity between a question and a candidate is then computed using cosine similarity (Manning et al., 2008):

$$\textit{Cosine Similarity}\ (q, c) = \frac{(q \cdot c)}{(||q|| \times ||c||)}$$

where q is the question embedding and c is the verbalized candidate embedding. Candidates are ranked by descending score and the top-k results returned.

Multiple encoders were evaluated before settling on BAAI/bge-large-en-v1.5. ICEWS has mostly short and formulaic quadruples, where light encoder alternatives such as all-MiniLM-L6-v2 showed weaker semantic discrimination. BGE-large (BAAI, 2024) offered competitive performance. I also built the evaluation pipeline (run_reranker_eval.py) to measure encoder performance using Hit@10 and MRR metrics (Manning et al., 2008), and the retrieval bundle construction script (build_reranker_eval_bundle_vA.py) to develop candidates for evaluation. Additionally, I implemented the entity-indexed retrieval component (baseline_retriever.py) using exact match and substring fallback logic, and the ID mapping utility (id_mapping.py) for consistent entity and relation lookup across pipeline components.

My core learning goal was to understand whether semantic similarity alone is sufficient for temporal question answering, or whether structural and temporal signals are required for meaningful event-based retrieval. The answer to this question turned out to be both clear and surprising, which I reflect on in Section 4.

4. Reflection and Self-Evaluation

4.1 Reflection on Project Outcomes

We expected semantic reranking would play a crucial role, as the encoder would be able to distinguish between semantically different events. However, we discovered that the temporal factor was dominant, providing +0.257 Hit@10 improvement whereas encoder ranking on its own gave no aggregate gain over the filtered baseline. Looking at the ICEWS dataset it is clear that most of the datapoints are very short and formulaic, giving an encoder very little natural language to process and gather context from. Another thing that is apparent is that a lot of events were re-occurring with different relationships and can only be distinguished based on temporal information. This is consistent with the findings of Sharma et al. (2023), who demonstrate that temporal relevance should directly influence answer selection in TKGQA, particularly for implicit questions where temporal grounding is most critical.

A concrete example from ICEWS14 illustrates this clearly:

(Al Jazeera, Praise or endorse, Head of Government (Egypt), 2014-06-18) (Head of Government (Egypt), Refuse to ease administrative sanctions, Al Jazeera, 2014-06-25)

Here the relationship between the same two entities reversed within the same week. Without temporal information the system cannot correctly reason about which event is being asked about. An encoder might give more weightage to the entity and relation parts of the verbalized text, whereas the temporal knowledge graph will not make this mistake. Thus the encoder can miss out on giving temporal information as much weight as it deserves. This tells us that some problems require a simpler structured approach as compared to complex neural components.

TKGE fusion gave a small +0.027 Hit@10 gain on implicit questions only, but the MRR slightly decreased from 0.571 to 0.557. This means that temporal information increased the chances of having the gold answer in the top-k, but it did not really help rank the gold answer close to rank 1. The gain is too small to be conclusive at a sample size of only n=74, and therefore a larger evaluation set would be needed to reach a firm conclusion.

Overall it was discovered that semantic reranking on its own was insufficient and requires supporting mechanisms for meaningful retrieval. Applying general-purpose encoders to domain-specific structured data without fine-tuning has clear limitations, particularly when the verbalized text is too short and formulaic to carry enough semantic context (Reimers & Gurevych, 2019).

4.2 Personal Contribution and Learning

My core contribution was exploring and integrating the encoder reranking component. I implemented encoder_reranker.py, which contains the full EncoderReranker class handling verbalization of quadruples into natural language, embedding via BGE-large-en-v1.5, and cosine similarity scoring. I evaluated alternative encoders including all-MiniLM-L6-v2 before settling on BGE-large. I built the evaluation pipeline (run_reranker_eval.py) to measure encoder performance using Hit@10 and MRR, and the retrieval bundle construction script (build_reranker_eval_bundle_vA.py) to prepare candidate pools. I also implemented entity-indexed retrieval with substring fallback logic (baseline_retriever.py) and the ID mapping utility (id_mapping.py).

Beyond the final pipeline, I explored encoder fine-tuning approaches preserved in the experimental/ folder of the repository. I implemented two strategies — Multiple Negatives Ranking (MNR) loss (Henderson et al., 2017) and Triplet loss with hard negatives (Hermans et al., 2017) — to attempt domain adaptation of the encoder to ICEWS-style event text. These did not improve sufficiently over the frozen BGE-large baseline, likely because 123 manually constructed examples were too few to fine-tune a large encoder effectively. This reinforced an important lesson: domain adaptation of large pretrained models requires substantially more training data than was available in this setting (Reimers & Gurevych, 2019).

From a technical standpoint, I gained practical experience with sentence encoder evaluation, cosine similarity ranking, evaluation metric implementation, and modular pipeline design for NLP research.

4.3 Teamwork and Challenges

The team collaborated effectively through regular meetings of 2-3 times per month via Zoom and in person. GitHub versioning kept contributions trackable and Google Drive was used for shared writing, with WhatsApp for day-to-day coordination.

The main challenge was the integration phase — each member worked on a separate component and connecting them into one functioning pipeline required careful alignment of data formats and interfaces. Components that worked in isolation sometimes required adjustments when connected to the rest of the pipeline. In hindsight, establishing shared interface contracts and running integration tests earlier would have reduced debugging time significantly. Task distribution was not always perfectly balanced, and I would advocate more explicit load balancing discussions and earlier integration milestones in future projects.

References:

- Sharma, A., Saxena, A., Gupta, C., Kazemi, M., Talukdar, P., & Chakrabarti, S. (2023). TwiRGCN: Temporally weighted graph convolution for question answering over temporal knowledge graphs. arXiv:2210.06281.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. arXiv:1908.10084.

- Henderson, M., Al-Rfou, R., Strope, B., Sung, Y., Lukács, L., Guo, R., Kumar, S., Miklos, B., & Kurzweil, R. (2017). Efficient natural language response suggestion for Smart Reply. arXiv:1705.00652.
- Hermans, A., Beyer, L., & Leibe, B. (2017). In defense of the triplet loss for person re-identification. arXiv:1703.07737.
- Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J. E., Navigli, R., Neumaier, S., Ngonga Ngomo, A. C., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). *Knowledge Graphs*. arXiv:2003.02320.
- Leblay, J., & Chekol, M. W. (2018). *Deriving validity time in knowledge graph.* https://doi.org/10.1145/3184558.3191639
- Boschee, E., Lautenschlager, J., O'Brien, S., Shellman, S., Starz, J., & Ward, M. (2015). *ICEWS Coded Event Data*. https://doi.org/10.7910/DVN/28075
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. https://nlp.stanford.edu/IR-book/
- BAAI. (2024). bge-large-en-v1.5 (model card). *Hugging Face*. https://huggingface.co/BAAI/bge-large-en-v1.5